Design Issues for Subprograms and Local Referencing Environment

Dr. Harish D. Gadade (डॉ. हरिष डी. गडदे)
Dept. of Computer Science and Engg.
COEP Technological University, Pune

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based?
6. Can parameters be defaulted?
7. What is the scope of local variables?
8. How are subprograms implemented?
9. Can subprograms be nested?

Are Subprograms required to have a Return Type?

No, **subprograms are not always required to have a return type**. It depends on the kind of subprogram:

Functions – These subprograms **must return a value**, so they require a return type.

Procedures / Subroutines – These subprograms **do not return a value**, so they **do not need a return type**.

```
int add(int a, int b)
{
    return a + b;
}
```

```
procedure DisplayMessage;
begin
    writeln('Hello, world!');
end;
```

So, whether a return type is required depends on whether the subprogram is expected to produce a value or just perform an action.

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
    - Pass By Value
    - Pass by Reference
    - Pass by Result
    - Pass by Value-Result
    - Pass by Name

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?

```
int area(int side) { return side * side; }
int area(int l, int w) { return l * w; }
```

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?

```
int fact(int n)
{
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based?

Positional or Keyword Parameters

1. Positional Parameters

```
#Positional Argument
def display(name,age):
  print("Name = ",name, "\nAge = ",age)

display("Ahaan",6)
```

```
#Positional Argument
def display(name,age):
  print("Name = ",name, "\nAge = ",age)

display("Ahaan")
```

```
#Positional Argument
def display(name,age):
  print("Name = ",name, "\nAge = ",age)

display(6,"Ahaan")
```

```
TypeError: display() missing 1 required
positional argument: 'age'
```

- **Languages:** C, C++, Java, Fortran, Pascal.

Positional or Keyword Parameters

2. Keyword Parameters

- Actual parameter is associated with the formal parameter by **name**.
- **Order does not matter.**
- Increases readability and reduces errors.
- **Languages:** Python, Ada, Kotlin, Swift.

```
#Keyword Arguments
def display(name,age):
  print("Name = ",name, "\nAge = ",age)

display(age=6,name="Ahaan")
```

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based?
6. Can parameters be defaulted?

Default Parameters

Default Parameters

- A **default parameter** is a parameter that automatically takes a predefined value if the caller omits it.
- Increases **flexibility** and reduces the need for multiple overloaded subprograms.

```
def display(name,msg="You are Cool"):
  print(name,msg)
display('Vihaan')
```

```
def area(pi=3.14,r=1):
  a=pi*r*r
  return a
a=area(r=2)
print(a)
```

```
def display(name,msg="You are Cool"):
  print(name,msg)
display('Vihaan',"Welcome")
```

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based, or both?
6. Can parameters be defaulted?
7. What is the scope of local variables?

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based, or both?
6. Can parameters be defaulted?
7. What is the scope of local variables?
8. How are subprograms implemented?

Implementation of Subprograms

Implementation of Subprograms

Every subprogram implementation needs to handle:

1. **Call/Return Mechanism** – how control is transferred.
2. **Parameter Passing** – how actual parameters are mapped to formals.
3. **Storage Allocation** – how local variables and state are stored.
4. **Access to Non-local Variables** – how variables outside the subprogram are referenced.
5. **Recursion Handling** – stack management for multiple activations.

Design Issues for Subprograms

Understanding these issues helps language designers and developers create clear, efficient, and safe subprograms.

1. Are subprograms required to have a return type?
2. What parameter passing methods are supported?
3. Can subprograms be overloaded?
4. Can subprograms be recursive?
5. Are formal parameters positional, keyword-based, or both?
6. Can parameters be defaulted?
7. What is the scope of local variables?
8. How are subprograms implemented?
9. Can subprograms be nested?

Can Subprograms be Nested?

Nested Subprogram

Yes, subprograms can be nested in many PPLs (Pascal, Ada, Python, JS, Lisp, etc.).

No nesting in C, early C++, Java methods (but lambdas/local classes give similar behavior).

A **nested subprogram** is a subprogram (function/procedure) defined **inside another subprogram**.

Local Referencing Environments

A local referencing environment defines the set of visible variables and their bindings (values) at any point in a subprogram.

1. What is Referencing Environment?
2. Types of Variable Access in Subprograms
3. Static vs Dynamic Scoping

Local Referencing Environments

Definition:

A **local referencing environment** of a subprogram is the set of **all names (variables, constants, parameters, etc.) that the subprogram can directly access** during its execution.

It Includes

- Local Variables
- Parameters
- Global Variables

```
x = 10  # Global variable
def outer():
    y = 20  # Local to outer
    def inner(z):  # Nested function
        # Local referencing environment of inner:
        # 1. z (parameter)
        # 2. y (non-local from outer)
        # 3. x (global)
        print(x + y + z)
    inner(5)
outer()
```

Types of Variable Access in Subprograms

In subprograms, variables can be classified based on **scope, lifetime, and visibility**. Here's a clear breakdown:

- Local Variables
- Parameters
- Global Variables
- Non Local Variables

```
x = 10  # Global variable
def outer():
    y = 20  # Local to outer
    def inner(z):  # Nested function
        # Local referencing environment of inner:
        # 1. z (parameter)
        # 2. y (non-local from outer)
        # 3. x (global)
        print(x + y + z)
    inner(5)
outer()
```

Static Vs Dynamic Scoping

1. Static (Lexical) Scoping

- **Definition:** The scope of a variable is determined **at compile time** based on the **program's textual layout**.

2. Dynamic Scoping

- **Definition:** The scope of a variable is determined **at runtime** based on **the calling sequence of subprograms**.